

Polynésie – 2025 – sujet1 - Correction

Exercice 2 (6 points)

👉 Remarque: il n'y avait pas de partie B dans cet exercice...

1. Fonction `possible_avec_penalites_seules`

Un score est possible uniquement avec des pénalités si c'est un multiple de 3.

```
def possible_avec_penalites_seules(score):  
    if score == 0:  
        return True  
    if score < 0:  
        return False  
    return possible_avec_penalites_seules(score - 3)
```

👉 Commentaire : on enlève 3 à chaque appel récursif jusqu'à atteindre 0 (succès) ou un nombre négatif (échec).

2. Tableau des solutions de 0 à 10

On autorise les scores : +3, +5, +7.

score	liste des solutions	nombre
0	[0]	1
1	[]	0
2	[]	0
3	[0,3]	1
4	[]	0
5	[0,5]	1
6	[0,3,6]	1
7	[0,7]	1
8	[0,5,8], [0,3,8]	2
9	[0,3,6,9]	1
10	[0,3,10], [0,5,10]	2

👉 Commentaire : on construit chaque score à partir de score -3, -5 ou -7.

3. Vérification de la relation pour $n = 10$

Formule :

$$f(n) = f(n-3) + f(n-5) + f(n-7) \quad f(n) = f(n-3) + f(n-5) + f(n-7)$$

Pour $n = 10$:

- $f(7) = 1$
- $f(5) = 1$
- $f(3) = 1$

Donc :

$$f(10) = 1 + 1 + 1 = 3$$

Or d'après le tableau nous avons trouvé 2 solutions.

Il manque une solution : [0,7,10] (7 + 3)

Donc :

[0,3,10]

[0,5,10]

[0,7,10]

$$f(10) = 3 \quad \checkmark$$

👉 *Commentaire : attention à ne pas oublier les combinaisons dans un ordre différent.*

4. Cas de base pour n de 0 à 6

- $f(0) = 1$
- $f(1) = 0$
- $f(2) = 0$
- $f(3) = 1$
- $f(4) = 0$
- $f(5) = 1$
- $f(6) = 1$

👉 *Commentaire : ces valeurs servent d'ancrage à la récursion.*

5. Fonction récursive `nb_solutions`

```
def nb_solutions(n):  
    if n < 0:  
        return 0  
    if n == 0:  
        return 1  
    if n == 1 or n == 2:  
        return 0  
    if n == 3:  
        return 1  
    if n == 4:  
        return 0  
    if n == 5:  
        return 1  
    if n == 6:  
        return 1  
    return nb_solutions(n-3) + nb_solutions(n-5) + nb_solutions(n-7)
```

👉 *Commentaire : pour $n > 6$ on applique directement la relation de récurrence.*

6. Méthode d'optimisation

👉 **Mémoïsation** (programmation dynamique).

👉 *Commentaire : on stocke les résultats déjà calculés pour éviter les appels récursifs redondants.*

7. Lignes utiles pour construire `solutions_possibles(11)`

On utilise :

- `solutions(11-3) = solutions(8)`
- `solutions(11-5) = solutions(6)`
- `solutions(11-7) = solutions(4)`

Donc on utilise les lignes :

- score 8

- score 6
- score 4

👉 *Commentaire : on applique la relation de récurrence sur les listes.*

8. Fonction `solutions_possibles`

Le code tel qu'il est proposé ne peut pas fonctionner. On propose :

```
def solutions_possibles(n):  
    if n < 0:  
        return []  
    if n == 0:  
        return [[0]]  
  
    solutions = []  
  
    for s in solutions_possibles(n-3):  
        solutions.append(s + [n])  
  
    for s in solutions_possibles(n-5):  
        solutions.append(s + [n])  
  
    for s in solutions_possibles(n-7):  
        solutions.append(s + [n])  
  
    return solutions
```

👉 *Commentaire : on prolonge chaque solution de $n-3$, $n-5$ et $n-7$ en ajoutant le score courant.*
